Lab: Store Terraform State in Remote Storage (Azure Storage Account)

Terraform keeps track of your infrastructure in a **state file**. By default, this file lives locally on your machine—but that can cause problems when working in teams (conflicts, lost state, no locking).

The solution? **Remote state storage**. In this lab, you'll configure Terraform to store its state file in an **Azure Storage Account**, giving you:

- Better collaboration
- Automatic state locking
- Reliable persistence

Learning Objectives

By the end of this lab, you will be able to:

1. Create an **Azure Storage Account** and **Blob Container** for remote state.
2. Configure **Terraform backend** to use Azure Storage.
3. Run Terraform commands with a **remote backend**.
4. Verify and manage the state file stored in Azure.

Prerequisites

Make sure you have:

- An **Azure subscription**
- **Terraform** installed locally
- **Azure CLI** installed and logged in (az login)

- Basic knowledge of Terraform syntax

Step 1: Set Up Azure Storage for Terraform State

Terraform needs a storage location that lives outside your project. Let's set that up.

1.1 Create a Resource Group

Update the name and location to match your needs.

```
az group create --name devops-week5-storage-rg --location "West US 2"
```

1.2 Create a Storage Account

The storage account will hold your state file.

```
az storage account create \
  --name devopsweekstorage1006 \
  --resource-group devops-week5-storage-rg \
  --location "West US 2" \
  --sku Standard_LRS
```

1.3 Create a Blob Container

This container will store the state file.

```
az storage container create \
  --name terraformstate \
  --account-name devopsweekstorage1006
```

1.4 Get the Storage Account Key

You'll need this key when configuring the backend.

```
az storage account keys list \
  --resource-group devops-week5-storage-rg \
```

```
  --account-name devopsweekstorage1006 \
  --query "[0].value" --output tsv
```

Copy the key—you'll use it soon.

Step 2: Configure Terraform Backend

Now that Azure is ready, let's configure Terraform to use it.

2.1 Create providers.tf

This file tells Terraform where to store state and which provider to use.

Replace values (resource group, storage account, subscription ID) with your own.

```
terraform {
  backend "azurerm" {
    resource_group_name  = "devops-week5-storage-rg"
    storage_account_name = "devopsweekstorage1006"
    container_name       = "terraformstate"
    key                  = "terraform.tfstate"
  }

  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "4.26.0"
    }
  }
}

provider "azurerm" {
  features {}
  subscription_id = "<your subscription>"
}
```

Step 3: Add Terraform Config Files

3.1 Define Variables – variables.tf

```
variable "resource_group_name" {
  type        = string
  description = "Name of the resource group"
}

variable "location" {
  type        = string
  description = "Azure location"
  default     = "West Europe"
}

variable "tags" {
  type        = any
  description = "Tags for resources"
}
```

3.2 Create a Resource Group – resource_group.tf

```
resource "azurerm_resource_group" "resource_group" {
  name     = var.resource_group_name
  location = var.location
  tags     = var.tags
}
```

3.3 Set Values – terraform.tfvars

```
resource_group_name = "devops-week5-rg-saurabh"
location            = "West Europe"
tags = {
  environment = "test"
}
```

Step 4: Run Terraform Commands

Initialize Terraform

```
terraform init
```

This configures Terraform to use the remote backend.

Validate Config

```
terraform validate
```

Preview the Plan

```
terraform plan
```

Notice the line:

> *Acquiring state lock*

Terraform locks the state to prevent conflicts. Once finished, it releases the lock.

Apply Changes

```
terraform apply -auto-approve
```

Step 5: Verify Remote State in Azure

Check the blob container to confirm the state file exists:

```
az storage blob list \
  --container-name terraformstate \
  --account-name devopsweekstorage1006 \
  --output table
```

Or, head to the **Azure Portal** → Storage Account → Container → terraform.tfstate.

Step 6: Explore the State File

Run some state commands:

```
terraform state list
```

```
terraform state show azurerm_resource_group.resource_group
```

Step 7: Clean Up Resources

When you're done, destroy everything to avoid costs:

```
terraform destroy --auto-approve
```

Lab Complete!

You've successfully:

- Set up **Azure Storage** for remote state
- Configured Terraform to use a **remote backend**
- Verified locking, persistence, and collaboration features